

KT-WinDiagTool

Windows Diagnostic Collection Tool for Security Products

A zero-configuration, admin-level diagnostics collector that gathers system state, traces, logs, registry keys, network data, and crash artifacts into a single ZIP bundle for support and field troubleshooting.

Language: C++17 · **Platform:** Windows 10+ · **Build:** Visual Studio 2026

Agenda

1. What is KT-WinDiagTool?
2. Architecture Overview
3. Configuration System (product JSON)
4. CLI Reference — Parameters & Options
5. Workflow Modes with Examples
6. Collector Deep Dive (19 collectors)
7. Analysis Pipeline — Crash Dumps
8. Output — HTML Report & ZIP Package
9. Field Troubleshooting Guide

What is KT-WinDiagTool?

KT-WinDiagTool is a **Windows-native diagnostic collector** designed for support engineers troubleshooting issues in security products (AV, firewall, DLP, EDR).

Key design goals:

- Single executable — no installation required
- One-command full collection: `KT-WinDiagTool.exe --config product.json`
- Runs as admin with UAC manifest embedded
- Product-agnostic via JSON config — point it at any product
- Dual mode: CLI for automation / programmatic launch, Win32 GUI for field use
- All 19 collectors run concurrently (`std::async`)
- Output: per-collector files + HTML summary + ZIP archive + `result.json`
- **Programmatic-launch-safe:** pipe-aware I/O, `--stop-event`, `--timeout`, machine-readable exit code + JSON result

Architecture: Collector Engine → 19 Collectors → ReportGenerator → ZipPackager

Architecture Overview

```
wWinMain
├── CLI Mode (--config / --cli / --repro flags detected)
│   ├── CLIHandler::ParseArgs()    Parse & validate all arguments
│   ├── [optional] ReproOrchestrator Start WPP + ETW + Perf + Packets
│   ├── CollectorEngine::RunAll()   Run all 19 collectors via std::async
│   │   ├── EnvVarCollector
│   │   ├── RegistryCollector
│   │   ├── ... (17 more)
│   │   └── CrashDumpCollector
│   ├── [optional] DumpAnalyzer     DbgHelp analysis of .dmp files
│   ├── ReportGenerator             HTML summary report
│   └── ZipPackager                  Bundle everything into .zip
└── GUI Mode (no CLI flags)
    └── Application::Run()           Win32 GUI with progress dialogs
```

Output: Desktop\KT-WinDiag\ (or --output <path>) → ZIP delivered to support

Configuration System — product.json

All product-specific parameters are in a **JSON config file**. One file per product.

```
{
  "product_name": "ExampleAV",
  "service_names": ["ExAVSvc", "ExAVDriver"],
  "log_paths": ["%ProgramData%\ExampleAV\Logs"],
  "install_log_paths": ["%TEMP%\ExampleAV_Install.log"],
  "registry_keys": [
    "HKLM\\SOFTWARE\\ExampleAV",
    "HKLM\\SYSTEM\\CurrentControlSet\\Services\\ExAVSvc"
  ],
  "event_log_channels": ["Application", "System",
    "Microsoft-Windows-ExampleAV/Operational"],
  "wpp_providers": [
    { "guid": "{12345678-...}", "name": "ExAV.Core", "level": 5, "flags": "0xFFFF" }
  ],
  "etw_providers": [{ "guid": "{AABBCCDD-...}", "name": "ExampleAV-ETW" }],
  "perf_counters": ["\\Process(ExAVSvc)\\*", "\\Memory\\*"],
  "crash_dump_paths": ["%ProgramData%\ExampleAV\CrashDumps"],
  "packet_capture": { "duration_seconds": 60 },
  "wpp_trace_levels": { "available": ["Critical", "Error", "Warning", "Info", "Verbose"],
    "default": "Warning" }
}
```

%ENV_VAR% **expansion** is supported in all path values.

CLI Reference — Required Parameter

Parameter	Description
<code>--config <path></code>	Required. Path to the product JSON config file.

```
# Minimum viable command – runs all collectors, output to Desktop\KT-WinDiag
KT-WinDiagTool.exe --config config\sample_product.json
```

Without `--config` the tool exits with code 2 (critical failure).

CLI Reference — Collection Options

Parameter	Default	Description
<code>--output <path></code>	Desktop\KT-WinDiag_YYYYMMDD_HHMMSS_<PID>	Output directory for all collected files
<code>--collectors <list></code>	all	Comma-separated collector IDs to run, or all
<code>--quiet</code>	off	Suppress all console output (useful in scripts)

Available collector IDs:

```
env      registry  installed  productlogs  installlogs
services proxy      network   firewall     userinfo
disk     wer        sysinfo   eventlog     etw
wpp      perf      packets   dumps
```

```
# Run only system context collectors
KT-WinDiagTool.exe --config myav.json --collectors sysinfo,env,registry,services

# Run everything, quiet, custom output path
KT-WinDiagTool.exe --config myav.json --output C:\SupportCase\12345 --quiet
```

CLI Reference — Tracing Options

Parameter	Default	Description
<code>--wpp-level <1-5></code>	3	WPP trace verbosity: 1=Critical, 2=Error, 3=Warning, 4=Info, 5=Verbose
<code>--product-log-level <lvl></code>	—	Set product's own log verbosity via registry before collection
<code>--enable-wpp</code>	off	Start WPP tracing before collection begins, stop after
<code>--disable-wpp</code>	off	Stop any active WPP trace sessions
<code>--enable-perf</code>	off	Start Performance Monitor logging before collection, stop after
<code>--disable-perf</code>	off	Stop any active perf logging sessions
<code>--duration <seconds></code>	30	Override capture duration for ETW, WPP, and packet capture (all timed collectors)

```
# Enable verbose WPP + perf logging during collection
KT-WinDiagTool.exe --config myav.json --enable-wpp --wpp-level 5 --enable-perf

# Raise product log verbosity before collecting
KT-WinDiagTool.exe --config myav.json --product-log-level Debug
```


CLI Reference — Analysis & Repro Options

Parameter	Description
<code>--repro</code>	Repro mode: start all traces, wait for Ctrl+C or <code>--stop-event</code> , stop traces, then collect
<code>--stop-event <name></code>	Named Win32 event for programmatic repro stop (alternative to Ctrl+C)
<code>--timeout <seconds></code>	Hard wall-clock limit on total collection; cancels collectors if exceeded
<code>--analyze-dumps</code>	After collection, run DbgHelp + optional WinDbg/cdb analysis on .dmp files
<code>--version</code>	Print tool version and exit (returns 0)
<code>--help / -h</code>	Show help and exit (returns 0)

Exit codes and machine-readable output:

Code	Meaning
0	All collectors succeeded
1	Partial success (some collectors failed)
2	Critical failure (config missing, bad argument, all collectors failed)

`result.json` is always written to the output directory — contains per-collector pass/fail, elapsed time, and error details for programmatic callers

Programmatic Launch — Automation & Integration

KT-WinDiagTool is designed to be launched as a subprocess by another product or automation harness with full API-like control.

I/O model when launched via `CreateProcess` :

- If parent passes `STARTF_USESTDHANDLES` (piped stdin/stdout/stderr), the tool detects `FILE_TYPE_PIPE` on the inherited handles and wires CRT directly to them — **no console window opens**.
- If launched from an existing terminal, `AttachConsole(ATTACH_PARENT_PROCESS)` is used.
- `MessageBoxW` dialogs are suppressed in CLI/pipe mode; errors go to `stderr`.

Programmatic repro stop (no Ctrl+C required):

```
# Parent creates event, passes name; signals when issue is reproduced
KT-WinDiagTool.exe --config myav.json --repro --stop-event MyProduct_DiagStop
# Parent calls: SetEvent(OpenEventW(..., L"MyProduct_DiagStop"))
```

Bounded collection with timeout:

```
KT-WinDiagTool.exe --config myav.json --timeout 120 --quiet
```

Reading results programmatically:

Workflow Example 1 — Full Standard Collection

```
KT-WinDiagTool.exe --config config\ExampleAV.json
```

What happens:

1. Loads and validates `ExampleAV.json`
2. Creates `Desktop\KT-WinDiag_20260221_143022_4512\`
3. Runs all 19 collectors concurrently
4. Prints per-collector pass/fail with timing
5. Generates `summary.html` and `result.json`
6. Packages everything into `ExampleAV_KTdiag_20260221_143022.zip`

```
[1/19] SystemInfo...      [OK]  SystemInfo      (1.2s)
[2/19] EnvironmentVars... [OK]  EnvironmentVars  (0.1s)
...
[19/19] CrashDumps...     [OK]  CrashDumps       (0.4s)

Passed: 19   Failed: 0
Output: C:\Users\Admin\Desktop\KT-WinDiag_20260221_143022_4512
ZIP:    C:\Users\Admin\Desktop\ExampleAV_KTdiag_20260221_143022.zip
Log:    %TEMP%\KT-WinDiagTool_20260221_143022_4512.log
```

Workflow Example 2 — Targeted Collection

For a fast, focused collection when the full set isn't needed:

```
# Connectivity issue: network + firewall + proxy + packet capture (60s)
KT-WinDiagTool.exe --config ExampleAV.json ^
  --collectors network,firewall,proxy,packets ^
  --duration 60

# Service crash investigation: services + registry + event logs + crash dumps
KT-WinDiagTool.exe --config ExampleAV.json ^
  --collectors services,registry,eventlog,dumps ^
  --analyze-dumps

# Performance degradation: perf counters + ETW + WPP at verbose level
KT-WinDiagTool.exe --config ExampleAV.json ^
  --collectors perf,etw,wpp,sysinfo ^
  --wpp-level 5
```

Workflow Example 3 — Repro Mode

Interactive (user-driven):

```
KT-WinDiagTool.exe --config ExampleAV.json --repro --wpp-level 5
```

Programmatic (automation-driven via named Win32 event):

```
KT-WinDiagTool.exe --config ExampleAV.json --repro --wpp-level 5 ^  
  --stop-event MyProduct_DiagStop  
# Caller calls: SetEvent(OpenEventW(EVENT_MODIFY_STATE, FALSE, L"MyProduct_DiagStop"))
```

Sequence (both modes):

```
=== REPRO MODE ===  
  
[OK] WPP tracing started (level 5 – Verbose)  
[OK] ETW tracing started  
[OK] Performance logging started  
[OK] Packet capture started  
  
Reproduce the issue now. Press Ctrl+C (or signal stop event) to collect...  
  
^C ← user presses Ctrl+C after reproducing the issue  
  
Stopping traces...  
Traces stopped. Running collectors...  
[1/19] SystemInfo
```

Workflow Example 4 — Full Analysis Run

```
# Step 1: Collect everything including dumps + deep WPP traces
KT-WinDiagTool.exe --config ExampleAV.json ^
--enable-wpp --wpp-level 5 ^
--enable-perf ^
--output C:\Case\12345 ^
--analyze-dumps

# Step 2 (separate): Quick connectivity triage only
KT-WinDiagTool.exe --config ExampleAV.json ^
--collectors network,proxy,firewall,packets ^
--output C:\Case\12345-net
```

Output structure produced:

```
C:\Case\12345\
system_info.txt      env_vars.txt        registry_export.txt
services.txt        eventlogs\          etw\*.etl
wpp_traces\*.etl    perf\*.blg         packets\capture.etl
dumps\*.dmp         dump_analysis\      windbg_analysis\
wer_summary.txt      summary.html
ExampleAV_KTdiag_20260221.zip
```

Collector 1 — Environment Variables (env)

Collection mechanism:

Calls `GetEnvironmentStringsW()` to snapshot the full process environment block, then walks the double-null-terminated string list, writing every `NAME=VALUE` pair to `env_vars.txt`.

Data collected:

All environment variables visible to the running process: `PATH`, `TEMP`, `SystemRoot`, `USERPROFILE`, `PROCESSOR_ARCHITECTURE`, vendor-specific variables injected by the product (e.g., `EXAV_HOME`, `EXAV_CONFIG`), and any policy-set variables.

Field issues this data resolves:

Symptom	Env Var Signal
Product can't locate its own config/binaries	<code>PATH</code> missing product install dir
Agent cannot write temp files	<code>TEMP</code> / <code>TMP</code> pointing to non-existent or read-only path
Wrong product behaviour on 64-bit	<code>PROCESSOR_ARCHITEW6432</code> indicates WOW64 mismatch
Cloud sync interference	<code>OneDrive</code> , <code>APPDATA</code> redirected to network share
Silent feature toggle	Vendor env var set to disable feature (<code>EXAV_DISABLE_FW=1</code>)

Collector 2 — Registry (registry)

Collection mechanism:

For each key path in registry_keys config, calls RegOpenKeyExW then recursively walks the key tree via RegQueryInfoKeyW, RegEnumValueW, and RegEnumKeyExW (up to 32 levels deep). All value types handled: REG_SZ, REG_DWORD, REG_QWORD, REG_BINARY, REG_MULTI_SZ, REG_EXPAND_SZ. Output: registry_export.txt in regedit-style format with hex+decimal for numerics.

Data collected:

Product configuration hive, service parameters, driver settings, licensing keys, policy overrides, feature flags stored in the registry.

Field issues this data resolves:

Symptom	Registry Signal
Service won't start	ImagePath wrong or missing; Start value set to 4 (Disabled)
Wrong update server	ServerURL / UpdatePath reg value points to staging env
Feature disabled by admin	Group Policy reg key overrides product default
Self-protection not applying	Driver UpperFilters / LowerFilters missing or wrong
Licensing failure	LicenseKey / ProductID absent or corrupt

Collector 3 — Installed Products (installed)

Collection mechanism:

Uses the Windows Installer COM/API: `MsiEnumProductsW` + `MsiGetProductInfoW` to enumerate all MSI-registered products with name, version, publisher, install date, and install location. Also walks `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall` for EXE-based installers.

Data collected:

Complete list of installed software: version, publisher, install date. Both 64-bit and 32-bit (WOW64) registry hives are queried.

Field issues this data resolves:

Symptom	Installed Products Signal
Component conflict	Competing AV/EDR product installed alongside target
Version mismatch on customer vs. expected	Product at wrong version; outdated component
Partial installation	Only some components show up (e.g., driver missing)
.NET / MSVC Redistributable missing	Required runtime absent
Policy software interfering	Corporate MDM agent or another security suite present
Downgrade scenario	Old version of the product co-exists with new

Collector 4 — Product Logs (productlogs)

Collection mechanism:

For each path in `log_paths` config, recursively walks the directory and copies files matching extensions `.log` , `.txt` , `.evtx` , `.csv` , `.xml` . Source paths support `%ENV_VAR%` expansion. Output: `product_logs\` subdirectory preserving subfolder structure. Filters by file age (configurable).

Data collected:

Product's own operational log files: real-time protection events, update history, scan results, error logs, module-specific logs.

Field issues this data resolves:

Symptom	Product Log Signal
Sporadic detection failures	Log shows scanner process crash/restart cycle
Update not applying	Update log shows HTTP 403 or CRC mismatch on download
High CPU spikes	Scan log shows full-disk rescans triggered on every event
GUI not reflecting real-time status	Service log shows IPC pipe disconnect
Random reboots	Log shows unexpected service termination + watchdog restart
Threat not quarantined	Policy log shows "quarantine disabled" or path exclusion hit

Collector 5 — Install Logs (installlogs)

Collection mechanism:

Copies files from paths listed in `install_log_paths` config (typically `%TEMP%\*.log` and `%TEMP%\*.txt` pattern paths). Handles MSI verbose log files, custom setup bootstrapper logs, and rollback logs.

Data collected:

MSI installer logs, custom bootstrapper logs, setup-phase error detail, component installation sequence, rollback events.

Field issues this data resolves:

Symptom	Install Log Signal
Installation fails silently	MSI log shows <code>Error 1603</code> with failing action name
Rollback during upgrade	Log shows component sequence that triggered rollback
Driver won't load post-install	Install log shows PnP device setup failure
Clean uninstall fails	Residual files/keys blocking reinstall
Feature missing after install	Custom action skipped due to privilege issue
Antivirus blocking own installer	Installer log shows access denied on self-extraction

Collector 6 — Services (services)

Collection mechanism:

Opens the Service Control Manager via `OpenSCManagerW(SC_MANAGER_ENUMERATE_SERVICE)` . Calls `EnumServicesStatusExW` with `SERVICE_WIN32 | SERVICE_STATE_ALL` to get all services with their live state and PID. For each service, calls `QueryServiceConfigW` to retrieve binary path, start type (`Boot/System/Auto/Manual/Disabled`), and service account. Output: `services.txt` (all services) + `product_services.txt` (config-specified services with explicit NOT FOUND / WARNING flags).

Data collected:

Every Win32 service: name, display name, current state, PID, binary path, start type, service account. Product services highlighted with missing/stopped warnings.

Field issues this data resolves:

Symptom	Services Signal
Product not protecting	Core service <code>Stopped</code> or <code>Start Pending</code> (stuck)
Service fails to start	Start type <code>Disabled</code> or binary path wrong/missing
Running as wrong account	<code>SYSTEM</code> vs <code>LocalService</code> vs custom account mismatch
PID useful for correlating with ETW	PID cross-reference to identify process in traces

Collector 7 — Proxy Settings (proxy)

Collection mechanism:

Reads WinHTTP proxy from `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\Connections\WinHttpSettings` (binary blob, parsed). Reads WinInet proxy from `HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings : ProxyEnable , ProxyServer , ProxyOverride , AutoConfigURL` . Also invokes `netsh winhttp show proxy` (WinHTTP system-wide proxy) and detects WPAD / PAC file configuration.

Data collected:

System-wide WinHTTP proxy, per-user WinInet proxy, bypass list, PAC/WPAD URL, proxy authentication settings.

Field issues this data resolves:

Symptom	Proxy Signal
Cloud sync / telemetry not working	WinHTTP proxy not configured; update service can't reach cloud
Proxy bypasses not applied	Bypass list missing <code>*.internal.corp</code>
MITM certificate errors	Intercepting proxy doing SSL inspection without product trust
License activation fails	Activation server blocked by proxy policy

Collector 8 — Network Configuration (network)

Collection mechanism:

- **Adapters:** `GetAdaptersAddresses(AF_UNSPEC, GAA_FLAG_INCLUDE_GATEWAYS)` — enumerates all adapters with IPv4/IPv6 addresses, gateway, DNS servers, MAC, DHCP status, DNS suffix, operational status.
- **Routing Table:** `GetIpForwardTable` — full IPv4 routing table (dest, mask, gateway, metric, interface index).
- **Active Connections:** `GetExtendedTcpTable(TCP_TABLE_OWNER_PID_ALL)` + `GetExtendedUdpTable(UDP_TABLE_OWNER_PID)` — all TCP states (LISTEN/ESTABLISHED/TIME_WAIT...) and UDP endpoints, each with owning PID.
- **DNS config:** reads `HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters` for domain, search list. Reads network profiles (Public/Private/Domain) from registry.

Field issues this data resolves:

Symptom	Network Signal
DNS resolution failures	DNS server IPs incorrect, search list wrong
Adapter not communicating	<code>OperStatus: Down</code> ; IP address missing

Collector 9 — Firewall Configuration (firewall)

Collection mechanism:

Instantiates `INetFwPolicy2` COM interface (Windows Firewall with Advanced Security). Queries all three firewall profiles (Domain, Private, Public): enabled state, default inbound/outbound actions, notifications. Enumerates all firewall rules via `INetFwRules` collection: name, direction, action (Allow/Block), protocol, local ports, remote addresses, application path, group, enabled flag, profile mask.

Data collected:

Firewall profile states, all rules (enabled and disabled), per-application and per-port exceptions, group policies overriding firewall.

Field issues this data resolves:

Symptom	Firewall Signal
Product agent blocked	Inbound/outbound rule for agent binary set to Block
Update service unreachable	Port 443/80 rule missing for product service account
VPN split-tunnel causing issue	Rule scoped to wrong interface profile
Remote admin connection blocked	Management port rule absent or wrong scope

Collector 10 — User Info (userinfo)

Collection mechanism:

- `GetUserNameW` — current running user context
- `GetTokenInformation(TokenGroups)` — all SIDs in the process token
- `LookupAccountSidW` — resolves each SID to human-readable name
- `LookupPrivilegeNameW` + `GetTokenInformation(TokenPrivileges)` — enumerates all privileges and their enabled/disabled state
- `WTSQuerySessionInformationW` — active RDP/WTS sessions (session ID, username, client name, state)
- `NetUserGetInfo` — domain user account info (lockout, password expiry, account flags)

Data collected:

Running user, token group membership, privilege set, all active sessions, domain account flags.

Field issues this data resolves:

Symptom	User Info Signal
Admin features unavailable	User lacks <code>SeDebugPrivilege</code> or not in local Administrators

Collector 11 — Disk Space (disk)

Collection mechanism:

Calls `GetLogicalDriveStringsW` to enumerate all drive letters, then for each fixed drive (`GetDriveTypeW == DRIVE_FIXED`) calls `GetDiskFreeSpaceExW` to retrieve total bytes, free bytes, and bytes available to the caller. Outputs a formatted table with GB values and percentage used.

Data collected:

All fixed disk volumes: total capacity, free space, available space (quota-aware), percentage used.

Field issues this data resolves:

Symptom	Disk Signal
Product fails to quarantine files	Quarantine volume at 100% — no space for new items
Log rotation not working	Log volume full; new log writes fail silently
Update download fails	Temp/cache volume full; partial download left
Scan database corruption	Signature store volume exhausted mid-write
Install rollback	Insufficient space on system drive during MSI operation
Crash dump not created	WER dump volume full; OS silently drops new dumps

Collector 12 — Windows Error Reporting (wer)

Collection mechanism:

Scans `%ProgramData%\Microsoft\Windows\WER\ReportArchive` and `ReportQueue`. Each crash report folder contains a `Report.wer` file (INI-style: `EventType`, `AppName`, `ModName`, `ExceptionCode`, `ReportIdentifier`). Parses each report and string-matches `AppName` and folder name against the configured product name. Copies matching report folders to output. Also reads WER policy from `HKLM\SOFTWARE\Microsoft\Windows\Windows Error Reporting: Disabled`, `DontShowUI`, `Consent`, `LocalDumps` config.

Data collected:

All crash reports on the system (last 30 days), product-matched reports highlighted and copied in full, WER policy state, LocalDumps configuration.

Field issues this data resolves:

Symptom	WER Signal
Random product crashes	WER reports show which module faulted + exception code
Crash dumps not generated	WER <code>Disabled=1</code> or <code>LocalDumps</code> not configured
Crash loops	Multiple WER entries in short time span

Collector 13 — System Information (sysinfo)

Collection mechanism:

Multi-source collection:

- **OS version:** `RtlGetVersion` (ntdll, bypasses Win10+ App Compat lies) + `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion` for DisplayVersion, Build, UBR, EditionID
- **Hardware:** WMI `Win32_ComputerSystem` (manufacturer, model, RAM), `Win32_Processor` (name, cores, clock), `Win32_BIOS` (vendor, version, serial)
- **Machine identity:** `GetComputerNameExW` (NetBIOS, DNS hostname, FQDN)
- **Time:** `GetSystemTime` , `GetLocalTime` , `GetTimeZoneInformation` , `GetTickCount64` (uptime)
- **KBs installed:** WMI `Win32_QuickFixEngineering` , sorted by install date descending

Field issues this data resolves:

Symptom	System Info Signal
Feature requires Win11 minimum	Build number reveals actual OS version
Support matrix mismatch	ARM64 vs x64 architecture wrong binary deployed
Low memory causing instability	Total RAM below product minimum requirement
KB missing causing incompatibility	Missing security update or required hotfix

Collector 14 — Event Logs (eventlog)

Collection mechanism:

Uses the Windows Event Log API (`wevtapi.dll`):

- **Binary export:** `EvtExportLog(channel, XPath-filter, .evtx)` — exports native `.evtx` for each configured channel, openable in Event Viewer
- **Text export:** `EvtQuery` → `EvtNext` (batches of 100) → `EvtRender(EvtRenderEventXml)` — renders each event as XML, written to `.txt`
- **Filter:** Last 7 days via XPath: `*[System[TimeCreated[timediff(@SystemTime) <= 604800000]]]`

Channels from config (e.g., `Application`, `System`, `Microsoft-Windows-ExampleAV/Operational`). Channels not found are silently skipped (non-fatal).

Field issues this data resolves:

Symptom	Event Log Signal
Service crash at specific time	System log: <code>Service Control Manager</code> error event
Driver load failure at boot	System log: PnP/Driver events with error codes
Product-specific operational event	Custom channel: product's own structured events
...	...

Collector 15 — ETW Traces (etw)

Collection mechanism:

Uses the Event Tracing for Windows API:

- 1. **Combined session (KTDiag_ETW):** StartTraceW → EnableTraceEx2(TRACE_LEVEL_VERBOSE, MatchAnyKeyword=0) for built-in Microsoft-Windows-Kernel-File + Microsoft-Windows-Kernel-Disk providers, **plus** all product ETW GUIDs from config. Captures 30 seconds to KTDiag_combined.etl .
- 2. **Per-provider sessions:** One ETW session per product provider GUID, each capturing 30 seconds to <ProviderName>.etl .

Stale sessions are auto-detected and stopped before retry. RAll TraceSessionGuard ensures sessions stop on scope exit even if cancelled.

Field issues this data resolves:

Symptom	ETW Signal
File I/O storms causing high CPU/disk	Kernel-File provider shows which process/path is thrashing
Slow disk response	Kernel-Disk I/O latency and queue depth visible per operation
Product-specific subsystem failure	Product ETW provider logs internal state machine transitions
Race condition in product code	High precision QPC timestamps reveal ordering

Collector 16 — WPP Software Traces (wpp)

Collection mechanism:

Delegates to `WppController::Start()` which calls `StartTraceW` and `EnableTraceEx2` for each provider GUID in `wpp_providers` config. WPP (Windows Software Trace Preprocessor) is a legacy but widely used tracing system in kernel and user-mode security drivers. Captures to `.etl` files for the configured duration (default 30 seconds). Trace level applied globally from config default or `--wpp-level` CLI override (1-5). WPP `.etl` files require `tracefmt.exe` or Windows Performance Analyzer with TMF/PDB to decode.

Data collected:

Binary WPP trace events from product's kernel driver and user-mode components. Contains internal state transitions, decision logic, timing.

Field issues this data resolves:

Symptom	WPP Signal
Driver making wrong decision	WPP shows exact code path and policy decision
Performance bottleneck in driver	Timestamp delta between WPP events reveals latency
Kernel-mode exception precursor	WPP trace shows last actions before BSOD

Collector 17 — Performance Counters (perf)

Collection mechanism:

Delegates to `PerfController::Start()` which uses the **PDH (Performance Data Helper)** API:

- `PdhOpenQuery` → `PdhAddEnglishCounterW` for each counter in `perf_counters` config
- Samples at 1-second intervals for the capture duration (default 30 seconds)
- Output: `.blg` binary log file (openable in `perfmon.exe` or `relog` command)

Counter examples from config: `\Process(ExAVSvc)\*`, `\Memory\*`, `\Processor(*)\% Processor Time`, `\PhysicalDisk(*)\*`.

Data collected:

Time-series samples of CPU%, memory, disk I/O, process-specific counters — all at 1-second resolution.

Field issues this data resolves:

Symptom	Perf Counter Signal
High CPU complaint	<code>\Process(ExAVSvc)\% Processor Time</code> reveals sustained load
Memory leak over time	<code>\Process(ExAVSvc)\Private Bytes</code> trend growing monotonically

Collector 18 — Packet Capture (packets)

Collection mechanism:

Invokes `netsh trace start capture=yes traceFile=<path> maxSize=256 overwrite=yes persistent=no` via `CreateProcessW` with stdout/stderr piped back. Waits for the configured duration (default 60 seconds, overridable via `--duration`). Then calls `netsh trace stop` . Output: `capture.etl` — a Network Monitor (NM3) format capture. Handles "session already running" by stopping the stale session and retrying.

Filter hint from config is logged as advisory; actual filtering best done post-capture in Network Monitor or Wireshark (use `nmcap` or `etl2pcapng` to convert).

Field issues this data resolves:

Symptom	Packet Capture Signal
Cloud connection failure	TCP RST or timeout to product's cloud endpoint
SSL handshake failure	TLS alert record (decrypt with product's key if available)
DNS resolution issue	DNS query with NXDOMAIN / SERVFAIL response
Update download stall	TCP window size collapse or retransmissions
Proxy CONNECT tunnel blocked	HTTP 407 / 403 response visible in capture

Collector 19 — Crash Dumps (dumps)

Collection mechanism:

Multi-source scan with filtering:

- **Product dump paths** from config (%ProgramData%\ExampleAV\CrashDumps)
- **Windows Kernel Minidumps:** %SystemRoot%\Minidump*.dmp
- **WER user-mode dumps:** %LocalAppData%\CrashDumps*.dmp
- **Registry-configured dump dir:** reads path from crash_dump_config_registry key

Filters: **last 30 days** (by GetFileTime), **max 500 MB** per file (skips memory dumps). Uses fs::recursive_directory_iterator(skip_permission_denied) . Copies all qualifying .dmp files with label-suffixed names to prevent collisions. Writes dump_summary.txt .

Field issues this data resolves:

Symptom	Crash Dump Signal
Product crashing silently	Dump present but never reported (no WER crash dialog)
Kernel panic (BSOD)	Kernel minidump + bug check code in Minidump\
Service crash loop	Multiple product dumps in short window

Analysis Pipeline — Crash Dump Analysis (`--analyze-dumps`)

After `CrashDumpCollector` copies `.dmp` files, the `--analyze-dumps` flag triggers two analysis passes:

Pass 1: DbgHelp (built-in, always available)

`DumpAnalyzer::Analyze()` uses `MiniDumpReadDumpStream` (DbgHelp API):

- Reads `ExceptionStream` → exception code, exception address
- Reads `ModuleListStream` → faulting module name + base address
- Writes `<dumpname>_analysis.txt` per dump

```
Exception: 0xC0000005 (Access Violation) in ExAVSvc.exe
Faulting module: ExAVNetFilter.dll
```

Pass 2: WinDbg / cdb.exe (if installed)

`WinDbgAutomation` locates `cdb.exe` in WinDbg install paths. Runs:

```
cdb.exe -z <dump> -c ".symfix;.reload;!analyze -v;kP;q"
```

Output: full automatic analysis, call stack with parameters, thread list — written to

`<dumpname>_windbg.txt`. If `cdb.exe` not found: writes `windbg_not_found.txt` with install guidance

Output — HTML Report & ZIP Package

HTML Summary Report (`summary.html`)

Generated by `ReportGenerator` using pure C++ string building (no third-party HTML library):

- Collection timestamp, machine name, product name
- Per-collector results table: collector name, pass/fail, elapsed time, error message
- Summary: total passed / total failed
- Viewable in any browser — no server required

Machine-Readable Result (`result.json`)

Always written to the output directory for programmatic callers:

```
{ "product": "ExampleAV", "timestamp_utc": "2026-02-21T14:30:22Z",  
  "pid": 4512, "log_file": "KT-WinDiagTool_20260221_143022_4512.log",  
  "exit_code": 0,  
  "collectors": [{ "id": "etw", "name": "ETW Traces", "success": true, "elapsed_seconds": 32 }, ... ],  
  "summary": { "total": 19, "passed": 19, "failed": 0 } }
```

ZIP Archive

`ZipPackager` uses `minizip/zlib` (`FetchContent` dependency):

Field Troubleshooting Guide — Quick Collector Matrix

Issue Category	Recommended Collectors
Product not starting	<code>services</code> , <code>registry</code> , <code>eventlog</code> , <code>installlogs</code>
Performance / High CPU	<code>perf</code> , <code>etw</code> , <code>wpp</code> , <code>sysinfo</code>
Memory leak	<code>perf</code> , <code>dumps</code> , <code>sysinfo</code>
Crash / BSOD	<code>dumps</code> + <code>--analyze-dumps</code> , <code>wer</code> , <code>eventlog</code>
Connectivity failure	<code>network</code> , <code>proxy</code> , <code>firewall</code> , <code>packets</code>
Update / licensing failure	<code>proxy</code> , <code>network</code> , <code>registry</code> , <code>installlogs</code>
Log verbosity issue	<code>productlogs</code> , <code>wpp</code> + <code>--wpp-level 5</code>
Installation failure	<code>installlogs</code> , <code>registry</code> , <code>services</code> , <code>sysinfo</code>
Policy / GPO conflict	<code>registry</code> , <code>firewall</code> , <code>services</code>
Reproduce on-demand issue	<code>--repro</code> + all collectors (+ <code>--stop-event</code> for automation)
Automated collection by another product	<code>--quiet</code> <code>--timeout <N></code> + read <code>result.json</code>

Repro Mode — Technical Flow

```
CLIHandler::Execute()
├── ReproOrchestrator::StartRepro()
│   ├── WppController::Start()      → StartTraceW × N providers
│   ├── EtwController::Start()      → StartTraceW KTDiag_ETW session
│   ├── PerfController::Start()     → PdhOpenQuery + PdhAddEnglishCounterW
│   └── PacketCaptureController::Start() → netsh trace start capture=yes
├── SetConsoleCtrlHandler(ReproCtrlHandler)
│   └── Blocks on poll loop (200ms sleep):
│       ├── InterlockedCompareExchange(&g_reproStopRequested) ← Ctrl+C handler
│       └── WaitForSingleObject(hStopEvent, 0)                  ← --stop-event signal
├── ReproOrchestrator::StopRepro()
│   ├── WppController::Stop()      → ControlTraceW(STOP)
│   ├── EtwController::Stop()      → ControlTraceW(STOP)
│   ├── PerfController::Stop()     → PdhCloseQuery
│   └── PacketCaptureController::Stop() → netsh trace stop
└── CollectorEngine::RunAll()      → Normal collection (skip ETW/WPP/Packets
                                   if .etl already in outputDir)
```

Deduplication: `EtwLogCollector`, `WppLogCollector`, `PacketCaptureCollector` each check for existing `.etl` files and skip re-capture in repro mode — **traces captured during repro are preserved.**

Key Technical Design Decisions

Decision	Rationale
<code>std::async</code> for all collectors	Max parallelism — 19 collectors run concurrently, wall time ~= slowest single collector
<code>CancelToken</code> (<code>shared_ptr<atomic_bool></code>)	Cooperative cancellation thread-safe across async tasks
<code>RtlGetVersion</code> not <code>GetVersionEx</code>	<code>GetVersionEx</code> returns 6.2 on Win10 unless manifested; <code>RtlGetVersion</code> always returns true OS version
<code>EvtExportLog</code> + text export dual format	<code>.evtx</code> for Event Viewer power users; <code>.txt</code> for grep/quick scan in support
Minizip FetchContent not system DLL	No dependency on user's system; reproducible builds
<code>netsh trace</code> not raw NDIScap	<code>netsh trace</code> requires no additional installs on Win10+; NDIScap available out-of-box
WPP via ETW API directly	Avoids <code>logman.exe</code> subprocess fragility; full control over session lifecycle
RAII <code>TraceSessionGuard</code>	Guarantees ETW session stops on exception/cancellation — prevents session leaks

Summary

KT-WinDiagTool provides a complete, zero-install diagnostic collection solution:

- **19 collectors** covering system context, product state, logs, tracing, network, and crash artifacts
- **Repro mode** ensures traces are active during fault reproduction (Ctrl+C or `--stop-event`)
- **Concurrent execution** minimises total collection time; `--timeout` bounds wall time
- **Product-agnostic** JSON config — adapts to any security product without recompilation
- **Analysis pipeline** with DbgHelp + WinDbg automation for crash dump triage
- **Self-contained output** — HTML report + `result.json` + ZIP archive ready for email/upload
- **First-class programmatic launch** — pipe-safe I/O, no console popups, machine-readable JSON result

Typical support workflow

```
1. Customer runs:  KT-WinDiagTool.exe --config ExampleAV.json --repro
2. Reproduces issue → Ctrl+C
3. Sends:          ExampleAV_KTdiag_<timestamp>.zip
4. Engineer opens: summary.html → identify failed collectors
5. Engineer drills: specific collector output → root cause
```

Appendix — Collector ID Reference

Collector ID	Class Name	Output File(s)
env	EnvVarCollector	env_vars.txt
registry	RegistryCollector	registry_export.txt
installed	InstalledProductsCollector	installed_products.txt
productlogs	ProductLogCollector	product_logs*
installlogs	InstallLogCollector	install_logs*
services	ServicesCollector	services.txt , product_services.txt
proxy	ProxySettingsCollector	proxy_settings.txt
network	NetworkConfigCollector	network_config.txt , routing_table.txt , active_connections.txt
firewall	FirewallConfigCollector	firewall_config.txt
userinfo	UserInfoCollector	user_info.txt
disk	DiskSpaceCollector	disk_space.txt
wer	WerCollector	wer_summary.txt , wer_reports*
sysinfo	SystemInfoCollector	system_info.txt , installed_kbs.txt